Scripts Showcase

Operation fsoc: Case Study

Gurmann Basran

Revised 2026-06-15 (rev. 3).

1 What the project ships

A set of production monitoring and auto-healing scripts runs across the infrastructure and, since the most recent milestone, the workstation too. It is well over a thousand lines of Bash and it grows with the system. Each script has a specific detection target, an alert path, and a recovery action. This document describes each at the goal level only. What I specifically do not include: the code itself, the layer each script watches, the tool it is integrating with, the thresholds it triggers at, the specific invariants it verifies, and the mechanism it uses to alert. All of those details would help someone trying to evade detection against my specific deployment, and I do not think the educational value of publishing them outweighs that cost. What is preserved is the reasoning, which is what transfers to another project anyway.

2 Defensive-chain invariant checker

Some tools can be "running" without actually providing the protection they were deployed for. Their process is alive, the basic healthcheck passes, and yet a specific invariant that makes the protection meaningful has silently regressed. This script runs on a tight interval and tests the invariant directly: not "is the service responding?" but "is the service responding *correctly*, in a way that depends on configuration I care about?" When the answer is no, the script both alerts and applies the fix. The design lesson is that liveness checks and correctness checks are different checks; naive healthchecks answer the liveness question and miss the correctness question entirely.

3 Out-of-band perimeter monitor

Any script running on the perimeter device cannot alert if the perimeter device itself is the thing that is down. A single-host monitor creates a "who watches the watcher" failure: the operator finds out by not getting the morning digest, which is hours too late. The monitor for this component runs on a different physical host with independent reachability. The design lesson is that every safety-critical check has to run from a host that does not depend on the thing being checked.

4 Remote-access liveness monitor

Remote-access protocols can be "up" in the sense that the daemon is listening while the actual connections from trusted devices have silently gone away. This script reads the connection-state telemetry through the component's API and alerts if any expected peer has not completed activity within a defined freshness window. The design lesson is the same as the previous script: the correct

question is whether the protocol is *functioning at the level I care about*, not whether the daemon is running.

5 Privileged-access real-time notifier

Every successful privileged login on every host pushes a real-time notification to my phone with the user, the source IP, and a geo-lookup. Alerts fire during login, not during log review. If I am starting a session from a coffee shop I expect the alert; if the alert arrives while I am asleep that is a page-one incident and I want to know about it immediately, not in the morning. The design lesson is that real-time notification on successful authentication beats daily log review for every metric that matters, because the point of the alert is to catch the event while it is still happening.

6 Daily state digest

Once a day, the infrastructure produces a summary of the previous twenty-four hours: authenticated sessions per host, detection-layer alert counts by severity, behavioral-defense actions, certificate expiry countdowns, disk utilisation per volume, backup status per machine, ingestion health across the logging spine, pending update notices. The content is not actionable in real time but it is the state snapshot I want in my inbox every morning. The design lesson is that a digest that *fails* to arrive is itself a signal; a separate "digest did not generate" watchdog catches that failure, because silent scripts are worse than noisy ones.

7 Image update watcher (manual-apply)

Unattended image updates are disabled. The watcher runs in monitor-only mode and generates an alert when new images are available; I apply updates manually after reading the release notes. The design rationale is that supply-chain attacks against public image registries have happened more than once in recent years, and unattended updates convert every image in the registry into a one-hop attack surface. The design lesson is that the default "auto-update everything" posture trades a small, bounded cost (manual review time) for an unbounded one (a catastrophic supply-chain event), and I choose the bounded cost.

8 Supply-chain forensic sweep

The update watcher above is the standing posture; this is what runs when a supply-chain attack actually breaks in the wild. When a poisoned-package or poisoned-image campaign hits the news, the question I need answered fast, and with evidence, is "did this reach us?" The sweep walks every host and container against a curated, version-controlled list of indicators (attacker domains, callback addresses, the signatures of the poisoned artifacts), cross-checks anything I fetched locally against the upstream published hashes, and pushes every confirmed-malicious indicator straight into perimeter DNS and firewall blocks so the same campaign cannot land later. The design lesson is that "we were not affected" is only a credible sentence when it is the output of a re-runnable

host-by-host check against a maintained indicator list, not a feeling; and every confirmed indicator should become an enforced block in the same pass, not a note to action later.

9 Configuration-drift detector

A security control here depended on a static mapping: an internal name-resolution layer pointed at fixed per-service addresses. The catch is that those addresses are handed out dynamically and can be reshuffled whenever the services restart, at which point the mapping silently points at the wrong place and the control it underpins quietly stops working. This script continuously compares the declared mapping against live runtime state and alerts the moment they diverge, before the divergence does any damage. The design lesson is that any invariant resting on a dynamically-assigned identifier is a latent failure waiting for the next restart; either pin the identifier or run a reconciler that treats drift as an alertable condition.

10 Tamper-baseline guard

A tamper-detection watchdog compares live counts of sensitive objects (firewall rules, tunnel peers, resolver overrides) against an expected baseline and alerts when they differ. The problem is that every legitimate change I make also moves those counts, so without a forcing function the watchdog cries wolf on my own work until I am trained to ignore it, and alert fatigue is exactly how a real change slips past. This script makes "I changed this on purpose" a single explicit step that re-derives the expected counts from the live source of truth and redistributes the new baseline. The design lesson is that a change-detection alarm is only as useful as its ability to tell authorized change from tampering; bake the baseline refresh into the change workflow so the alarm stays meaningful instead of becoming noise.

11 Operator-intervention channel

During a long autonomous maintenance run, most steps are mechanical, but a handful need a human: approving a privileged action, doing a physical step, or confirming that an alert actually reached my phone. This is a dedicated two-way push channel for exactly those moments. The automation pings me with a templated message (what is needed, why, whether it is blocked, how to unblock it, and when), and I answer on the same channel, so the run does not silently stall at the first prompt and I am not chained to the keyboard for the parts that do not need me. The design lesson is that full autonomy is a fiction for fortress-grade work; design the human-in-the-loop in explicitly, with a crisp escalation channel, rather than pretending the run will never need a decision.

Since I first wrote this, that channel has become one piece of a larger orchestration layer that runs the maintenance process the way a delivery pipeline runs a release, with the risky steps gated. Each task is classified by the class of change it represents, and that class sets the governance it gets: trivial reversible work proceeds on its own, anything that touches the administrative plane or cannot be cleanly undone is held on this channel for an explicit approval, and a defined set of high-consequence change classes are hard stops that require a human no matter what. Every run

leaves a configuration-managed record of what it changed and why. The refinement to the design lesson is that the right amount of autonomy is not a global setting; it is a function of how reversible and how blast-prone a specific change is, so the human-in-the-loop belongs at the boundary of each change class rather than applied uniformly or skipped uniformly.

12 Detection-layer self-test

Most of a good monitoring fabric is silent by design; it only speaks when something is wrong. That creates a trap: silence could mean "everything is healthy" or it could mean "the detector died three weeks ago and nobody noticed." This script resolves the ambiguity on demand. It runs every monitor once and emits a unique, identifiable marker down each notification channel, so I can confirm end to end that every detection path and every subscription is genuinely alive. The design lesson is that silence-by-design is not the same as healthy; a detection layer you cannot prove is alive is one you do not actually have, so build a proof-of-life that exercises each channel distinctly.

13 Operational-health and capacity watchdog

Every other monitor in this set asks a security question or a correctness question. This one asks a different kind of question entirely: not "is this protection working" but "does this host have headroom, or is it quietly running out of something." It came out of an incident where a runaway process pinned a processor core for the better part of a week and nothing alerted, and another where a maintenance job exhausted a host's memory at idle, because the monitoring I had was all liveness and correctness and none of it was capacity. The watchdog runs on a tight interval, from a host that does not depend on the hosts it is watching, and looks for the signatures of resource exhaustion before they become an outage: a process consuming a processor for a sustained stretch rather than a brief spike, memory and swap pressure trending toward the edge, a filesystem or a scratch volume filling up. It is already deployed and watching the load-bearing hosts, with coverage extending to the remaining corners of the fleet as I bring each one under it. It is read-only and alert-only by design; it does not try to fix anything, because the right response to capacity pressure is a human decision about what to cut or grow, not an automated reflex. The design lesson is that liveness, correctness, and capacity are three different questions, and the third one is temporal: it cannot be answered by a snapshot, only by watching a trend over time, so it needs a check that keeps asking rather than a check that runs once.

14 Backup-recoverability prover

A backup that exists is not a backup that restores, and the only way to know which one you have is to actually decrypt and unpack it before you need it. This routine treats recoverability as something to prove rather than assume: it decrypts each archive in a bundle, confirms the contents are structurally intact rather than truncated or corrupt, and does this against real bundles on a schedule, so the question "can I actually recover from these" has a current, evidence-backed answer instead of a hopeful one. A companion check watches the backups themselves for staleness: if a fresh bundle does not show up inside the window it should have, that silence is treated as an

alertable failure, because a backup job that quietly stopped is exactly the kind of thing you discover at the worst possible moment otherwise. The design lesson is that the failure mode of a backup is silence; it keeps producing nothing, or producing garbage, and looks fine from the outside, so the only honest verification is to exercise the restore path and to alarm on the absence of fresh output, not to trust that "the job is configured" means "the job is working."

15 Access-policy self-check

When the fleet moved to short-lived signed certificates as the only accepted form of remote access, the risk was not the cutover itself; it was drift afterward. A long-lived static key quietly re-added for convenience, a weaker authentication path left enabled as a fallback, a configuration change that appended rather than replaced, any of these silently reopens the door the cutover was meant to close, and none of them throws an error. This check reads the live access configuration across the fleet and verifies the invariants directly: only the intended authentication method is accepted, no static-key fallback survives, the deploy mechanism replaced the policy rather than layering onto it. It is deliberately self-checking and non-additive: it fails loudly if it finds the access posture has drifted back toward something weaker than declared. The design lesson is that a security cutover is a moment, but a security posture is a state, and the only way a state stays true is a check that re-derives it from the live system and refuses to pass when reality and intent have diverged.

16 Self-healing workstation supervisor

A workstation only earns the title "daily driver" when it recovers from its own faults without me babysitting it. The most recent milestone gave the workstation a manifest that declares every desktop component's expected role, health check, resource budget, and rollback path; a validator that reads live state against that manifest; a constrained repair planner that may only take pre-approved corrective actions; supervised restarts of the user-session services; and a pre-flight gate with a known-good visual baseline to fall back to. The design lesson is that reliability is an engineered property and not a hope; encode the desired state as a checkable manifest, give the system a bounded set of safe self-repairs, and gate every change behind health and visual-regression checks so the self-healing can never make things worse than it found them.

17 Remote workstation wake

From a trusted remote context, a script on an always-on inside host wakes a physical workstation that is otherwise asleep. Combined with the encrypted outbound path described below, this is the "reach my desktop while I am away" workflow. The design lesson is that the power-on signal should originate from a host already trusted inside the perimeter, with the remote operator reaching that trusted host through existing authorized access; no inbound port to the workstation is ever exposed.

18 Outbound-initiated encrypted path

I want to reach my home network and my devices from anywhere, and I want to do it without exposing a single inbound port from the less-trusted side of the boundary. I built the first version of this as an application-layer reverse tunnel: a portable device held a persistent outbound connection, with automatic reconnection, to an always-on inside host, and the inside host could then reach the device back down that connection. It worked. I later retired it and replaced it with a modern encrypted tunnel that the device still initiates outbound, because the encrypted tunnel gives the same reachability with stronger cryptography, a cleaner trust model, and one fewer long-running daemon to babysit. The implementation changed; the principle did not, and it is probably my favourite pattern in the whole project: every path that has to exist should be outbound from the more-trusted side of the boundary, never inbound from the less-trusted side. Mobile devices reach home; home does not expose itself to the internet to make that work. The design lesson is that when a pattern is right, you keep the pattern and upgrade the mechanism under it; the same reasoning underpins modern zero-trust networking.

There is a sequel to this one. For a while the site also ran a second inbound path: a third-party-managed tunnel fronting one self-hosted service, with an identity gate in front of it. It worked, but it was a second way in that I had to trust, maintain, and reason about, for a service I had stopped really using. As part of a resource-recovery pass I retired the service and tore that tunnel down with it, which left the outbound-initiated tunnel as the single inbound mechanism for the whole site. The reduction is the point: every distinct entry path is attack surface and cognitive surface at the same time, and the cleanest way to harden a path is sometimes to prove you do not need it and then remove it. Tearing it down had its own discipline, because a tunnel like that has a connector that must deregister cleanly before the upstream resources are deleted, and shared pieces, a network and a credential, that something else might still depend on; getting that order wrong is its own class of trap, and it became one of the integration lessons in the companion document.

19 Design patterns common to every script

Some practices show up across every script in the set. None of them are innovations, but every one of them came from an incident where not doing it cost me something.

1. **Fail-loud execution.** Scripts abort on the first error, on unset variables, and on pipeline failures. No silent partial success.

2. **No inline credentials.** Every credential is pulled from an encrypted-at-rest file or a least-privilege API account; nothing is in the script text.

3. **Timeouts on every network call.** A script that hangs waiting for a dead endpoint is a script that will be noticed only after someone asks why the digest never arrived.

4. **Alert text is complete on its own.** A notification says what failed, what was expected, and what was observed. Future-me at 3 AM needs to understand what happened without opening the script.

5. **Idempotent auto-heal.** Fix actions are safe to run repeatedly against an already-healthy system.

6. **Least-privilege credentials per script.** Each script uses a different account, scoped to exactly what that script needs.

7. **Tested with deliberate breakage.** Every alert path was validated by manually triggering its failure condition before the script went into production. An untested alert path is an alert path that is not there.

8. **Proof-of-life over assumption.** A silent detector is assumed dead until proven otherwise; the self-test exists so I can prove the whole chain alive on demand rather than trusting that no news is good news.